

Python Programming

Complete Notes for Beginners — with a Data Science perspective

Class Notes

Table of Contents

1. What is Python
2. Python Installation and IDE
3. Variables
4. Python Datatypes
5. String Operations and Functions
6. String Formatting
7. Type Casting
8. Operators
9. Data Structures (List, Tuple, Set, Dict)
10. Inbuilt Functions
11. Common Errors

1. What is Python

Python is a high-level, general-purpose programming language. "High-level" means it is written in a way that is close to plain English, so the programmer does not need to deal with low-level details like memory management or hardware instructions. It was created by Guido van Rossum and first released in 1991. Today Python is one of the most widely used languages in the world, and it is the single most popular language in the data science and machine learning field.

Applications

Where Python is actually used, especially in Data Science:

- **Data Analysis:** cleaning, exploring, and summarizing large datasets using libraries like Pandas and NumPy
- **Machine Learning & AI:** building predictive models using Scikit-learn, TensorFlow, PyTorch
- **Data Visualization:** creating charts and dashboards using Matplotlib, Seaborn, Plotly
- **Web Development:** building websites and APIs using Django, Flask, FastAPI
- **Automation / Scripting:** automating repetitive tasks like file handling, report generation
- **Game Development:** simple 2D games using Pygame

Why Python is popular

Advantages

- Simple, readable syntax — beginner friendly
- Free and open source
- Huge collection of libraries for data science and AI
- Works on Windows, Mac, and Linux
- Massive community, easy to find help online

Limitations

- Slower than compiled languages like C++ or Java
- Not ideal for mobile app development
- Uses more memory compared to lower-level languages
- Global Interpreter Lock (GIL) limits true multi-threading

EXAMPLE 1 — FIRST PYTHON PROGRAM

```
print("Hello, Data Science World!")
```

Output:

Hello, Data Science World!

EXAMPLE 2 — PYTHON DOING A QUICK CALCULATION

```
total_students = 50
passed = 42
pass_percentage = (passed / total_students) * 100
print("Pass percentage:", pass_percentage)
```

Output:

Pass percentage: 84.0

Note: Almost every data science job today expects working knowledge of Python because most modern AI and analytics tools are built on top of it.

2. Python Installation and IDE

Before writing Python programs, you need two things: the Python interpreter installed on your computer, and a place (editor/IDE) to write and run your code.

Installing Python

1. Go to the official website: python.org
2. Download the latest version for your operating system
3. Run the installer and make sure to check "**Add Python to PATH**" before clicking Install
4. Verify installation by opening Command Prompt/Terminal

EXAMPLE 1 — CHECKING PYTHON VERSION

```
python --version
```

Output:

```
Python 3.12.0
```

EXAMPLE 2 — CHECKING INSTALLED PACKAGES (PIP)

```
pip --version
```

Output:

```
pip 24.0 from C:\Python312\lib\site-packages\pip
```

What is an IDE

An IDE (Integrated Development Environment) is software where you write, run, and debug your code, all in one place, instead of using separate tools.

Applications — choosing the right tool

Which tool is used where:

- **Jupyter Notebook / Google Colab:** the standard choice in data science — lets you run code in small chunks (cells) and see output, tables, and graphs instantly next to the code
- **VS Code / PyCharm:** best for building full applications, web apps, or large multi-file projects
- **IDLE:** good for absolute beginners just learning basic syntax

IDE / Editor	Best For
IDLE	Comes free with Python, good for beginners
VS Code	Lightweight, most popular, highly customizable
PyCharm	Full-featured, good for large software projects
Jupyter Notebook	Best for data science, step-by-step code with visible output
Google Colab	Runs in browser, no installation, free GPU access

Jupyter/Colab — Advantages

- See output (tables, graphs) right below the code
- Great for experimenting and testing small pieces
- Colab needs no installation, runs online for free

Jupyter/Colab — Limitations

- Not ideal for building large, production-level applications
- Harder to organize code into reusable modules

Tip: For learning Python basics use IDLE or VS Code. Once you start data analysis, switch to Jupyter Notebook or Google Colab — that is what most data scientists use daily.

3. Variables

A variable is a name given to a memory location that stores a value. Think of it as a labeled container that holds data, and that data can change while the program runs. In data science, variables are used constantly to hold datasets, column values, model results, and settings.

Rules for naming variables

- Must start with a letter or underscore (not a number)
- Can contain letters, numbers, and underscores
- Cannot use spaces or special symbols like @, #, %
- Case-sensitive: `name` and `Name` are different variables
- Cannot use Python keywords like `if`, `for`, `class` as variable names

Applications

How variables are used in data science:

- Storing a dataset after loading it: `data = pd.read_csv("sales.csv")`
- Storing model accuracy, thresholds, or hyperparameters
- Holding intermediate results during data cleaning

EXAMPLE 1 — BASIC VARIABLE ASSIGNMENT

```
name = "Rahul"
age = 20
city = "Nagpur"

print(name)
print(age)
print(city)
```

Output:

```
Rahul 20 Nagpur
```

EXAMPLE 2 — MULTIPLE ASSIGNMENT AND UPDATING A VARIABLE

```
a, b, c = 10, 20, 30
print(a, b, c)

score = 50
score = score + 10    # updating the variable
print(score)
```

Output:

10 20 30 60

Advantage of dynamic typing

- No need to declare datatype — faster to write code
- Same variable can hold different types over time

Limitation

- Can cause hidden bugs if a variable's type changes unexpectedly
- Slightly slower than statically typed languages internally

Note: Python is dynamically typed, meaning you don't need to declare the data type of a variable — Python figures it out automatically based on the value assigned.

4. Python Datatypes

A datatype tells Python what kind of value a variable is holding — a number, text, a list of values, and so on. Choosing the correct datatype matters a lot in data science because it affects memory usage, speed, and what operations you can perform on a dataset column.

Datatype	Description	Example
int	Whole numbers	10, -5, 200
float	Decimal numbers	3.14, -0.5
str	Text / string	"Hello"
bool	True or False values	True, False
list	Ordered, changeable collection	[1, 2, 3]
tuple	Ordered, unchangeable collection	(1, 2, 3)
set	Unordered, unique collection	{1, 2, 3}
dict	Key-value pairs	{"name": "Amit"}
NoneType	Represents no value / missing data	None

Applications

Datatypes in a data science context:

- **int / float:** used for numeric columns like age, price, salary — required for any mathematical or statistical calculation
- **str:** used for categorical columns like names, city, gender labels
- **bool:** used for flags/filters, e.g. "is_customer_active"
- **None:** represents missing values (similar to NaN in Pandas)

EXAMPLE 1 — CHECKING DATATYPE OF DIFFERENT VALUES

```
x = 25
y = "Python"
z = 3.5

print(type(x))
print(type(y))
print(type(z))
```

Output:

```
<class 'int'> <class 'str'> <class 'float'>
```

EXAMPLE 2 — BOOLEAN AND MISSING VALUE

```
is_pass = True
marks = None

print(type(is_pass))
print(type(marks))
print(marks is None)
```

Output:

```
<class 'bool'> <class 'NoneType'> True
```

Tip: Use the `type()` function anytime you want to check what kind of data a variable holds — very useful while cleaning a messy dataset.

5. String Operations and Functions

A string is a sequence of characters written inside single quotes `' '` or double quotes `" "`. Strings are extremely common in data science because most raw, real-world data — names, addresses, product categories, text reviews — arrives as text and needs to be cleaned before analysis.

Basic string operations

EXAMPLE 1 — CONCATENATION, REPETITION, INDEXING, SLICING

```
a = "Hello"
b = "World"

print(a + " " + b)    # concatenation
print(a * 3)          # repetition
print(len(a))         # length
print(a[0])           # indexing
print(a[1:4])         # slicing
```

Output:

```
Hello World HelloHelloHello 5 H ell
```

Common string functions

Function	Use
upper()	Converts to uppercase
lower()	Converts to lowercase
strip()	Removes extra spaces from start/end
replace(old, new)	Replaces part of the string
split()	Splits string into a list
find()	Finds position of a substring
join()	Joins list elements into a string

Applications

String handling in data cleaning (Data Science use case):

- Removing extra spaces from messy column entries: `" Delhi ".strip()`
- Standardizing text case so "delhi" and "Delhi" are treated as the same category
- Splitting a full name column into first name and last name
- Cleaning text data before feeding it into an NLP model

EXAMPLE 2 — CLEANING MESSY DATA-LIKE TEXT

```
raw_entry = "  Rahul Sharma  "

clean_name = raw_entry.strip().title()
print(clean_name)

full_name = "Rahul_Sharma"
parts = full_name.split("_")
print(parts)
```

Output:

```
Rahul Sharma ['Rahul', 'Sharma']
```

Advantages

- Very flexible for cleaning and reshaping text data
- Built-in functions cover most common cleaning tasks

Limitations

- Manual string cleaning gets slow on very large datasets — Pandas vectorized string methods are used instead
- Complex pattern matching needs regular expressions (regex), which have a learning curve

6. String Formatting

String formatting means inserting variable values inside a string neatly, instead of manually joining pieces of text with `+`. This is used constantly when printing reports, labels for charts, or messages that combine text and numbers.

1. %s formatting (old style)

EXAMPLE 1

```
name = "Riya"
age = 22
print("My name is %s and I am %d years old" % (name, age))
```

Output:

My name is Riya and I am 22 years old

EXAMPLE 2

```
score = 89.5
print("Your score is %.1f out of 100" % score)
```

Output:

Your score is 89.5 out of 100

2. .format() method

EXAMPLE 1

```
name = "Riya"
age = 22
print("My name is {} and I am {} years old".format(name, age))
```

Output:

My name is Riya and I am 22 years old

EXAMPLE 2

```
print("{0} scored {1} marks in {2}".format("Aman", 78, "Maths"))
```

Output:

```
Aman scored 78 marks in Maths
```

3. f-string (recommended, modern way)

EXAMPLE 1

```
name = "Riya"  
age = 22  
print(f"My name is {name} and I am {age} years old")
```

Output:

```
My name is Riya and I am 22 years old
```

EXAMPLE 2 — USING AN EXPRESSION INSIDE F-STRING

```
price = 250  
quantity = 3  
print(f"Total bill: {price * quantity}")
```

Output:

```
Total bill: 750
```

4. r-string (raw string)

Raw strings ignore escape sequences like `\n` or `\t`, printing them exactly as written. Used mostly for file paths and regex patterns — both very common when loading datasets or cleaning text.

EXAMPLE 1

```
path = r"C:\Users\new_folder\dataset.csv"  
print(path)
```

Output:

C:\Users\new_folder\dataset.csv

EXAMPLE 2

```
pattern = r"\d+" # regex pattern for digits, used later with the re module
print(pattern)
```

Output:

\d+

Applications

Where formatting is used in Data Science:

- Printing model results neatly: `f"Accuracy: {accuracy:.2f}%"`
- Generating dynamic chart titles or file names
- Building readable log messages while a script runs

Tip: f-strings are the most used method in modern Python because they are short, fast, and easy to read — prefer them over %s and .format() in new code.

7. Type Casting

Type casting means converting one datatype into another — for example, turning a piece of text into a number so you can do maths on it. This is extremely common while cleaning real datasets, where numbers are often loaded as text.

Function	Converts to
int()	Integer
float()	Decimal number
str()	String
list()	List
tuple()	Tuple

Applications

Type casting in Data Science:

- Converting a "price" column loaded as text (e.g. "500") into numbers before calculating totals or averages
- Converting a numeric ID column into a string when it should be treated as a category, not a quantity
- Converting a list of numbers into a tuple to use as a fixed key in a dictionary

EXAMPLE 1 — TEXT TO NUMBER

```
a = "25"
b = int(a)
print(b + 5)
print(type(b))
```

Output:

```
30 <class 'int'>
```

EXAMPLE 2 — NUMBER TO STRING, AND FLOAT TO INT

```
c = 10
d = str(c)
print(d + " apples")

price = 99.99
whole_price = int(price)
print(whole_price)
```

Output:

10 apples 99

Advantages

- Lets you fix datatype mismatches loaded from messy files (CSV, Excel)
- Needed before performing any numeric calculation on text-like columns

Limitations

- Converting `int()` on a decimal simply cuts off the decimal part, it does not round
- Wrong or non-numeric strings will crash the program if not handled

Warning: Converting a non-numeric string like `int("hello")` will give an error. Always check the data before converting, especially with real-world datasets.

8. Operators

Operators are symbols used to perform operations on values and variables — calculations, comparisons, and logical decisions. They are the building blocks of almost every condition and formula you will write, including statistical calculations in data science.

1. Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
//	Floor Division	5 // 2	2
%	Modulus (remainder)	5 % 2	1
**	Exponent (power)	5 ** 2	25

EXAMPLE 1

```
x = 10
y = 3
print(x + y, x - y, x * y, x / y)
```

Output:

13 7 30 3.3333333333333335

EXAMPLE 2 — USED IN A STATISTICS-STYLE CALCULATION

```
total_marks = 450
subjects = 5
average = total_marks / subjects
print("Average:", average)
```

Output:

Average: 90.0

2. Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 2	False
!=	Not equal to	5 != 2	True
>	Greater than	5 > 2	True
<	Less than	5 < 2	False
>=	Greater or equal	5 >= 5	True
<=	Less or equal	5 <= 2	False

EXAMPLE 1

```
marks = 85
print(marks >= 40)
```

Output:

True

EXAMPLE 2 — FILTERING-STYLE CHECK, COMMON IN DATA ANALYSIS

```
age = 17
print(age >= 18)
```

Output:

False

3. Logical Operators

Operator	Meaning	Example
and	True if both conditions are true	(5>2) and (3>1) → True
or	True if at least one condition is true	(5>2) or (3<1) → True
not	Reverses the result	not(5>2) → False

EXAMPLE 1

```
marks = 75
attendance = 80
print(marks >= 40 and attendance >= 75)
```

Output:

True

EXAMPLE 2

```
is_holiday = False
print(not is_holiday)
```

Output:

True

4. Assignment Operators

Operator	Example	Same as
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3

EXAMPLE 1

```
x = 10
x += 5
print(x)
```

Output:

15

EXAMPLE 2 — RUNNING TOTAL, LIKE ACCUMULATING A SUM IN A LOOP

```
total = 0
total += 20
total += 30
print(total)
```

Output:

50

Applications

Operators in Data Science:

- Arithmetic operators for calculating averages, totals, percentages, and normalizing data
- Comparison operators for filtering rows, e.g. selecting all customers with `age >= 18`
- Logical operators for combining multiple filter conditions, e.g. `marks >= 40 and attendance >= 75`

9. Data Structures (List, Tuple, Set, Dict)

Data structures are ways of organizing and storing multiple values together. Choosing the right structure is one of the most important skills in data science, because raw data almost always comes as a collection of many values, not a single one.

1. List — ordered, changeable, allows duplicates

EXAMPLE 1

```
fruits = ["apple", "banana", "mango"]
fruits.append("orange")
fruits.remove("banana")
print(fruits)
print(fruits[0])
```

Output:

```
['apple', 'mango', 'orange'] apple
```

EXAMPLE 2 — STORING NUMERIC DATA FOR CALCULATION

```
marks = [78, 85, 90, 62, 74]
print(sum(marks) / len(marks))
```

Output:

```
77.8
```

Common list functions: `append()` , `remove()` , `insert()` , `sort()` , `pop()` , `len()` , `reverse()`

Advantages

- Flexible — can add/remove items anytime
- Keeps order, good for sequential data like time-series values

Limitations

- Slower than NumPy arrays for large numeric datasets
- Allows duplicates, so not ideal when uniqueness matters

2. Tuple — ordered, unchangeable, allows duplicates

EXAMPLE 1

```
colors = ("red", "green", "blue")
print(colors[1])
print(len(colors))
```

Output:

```
green 3
```

EXAMPLE 2 — USING A TUPLE AS A FIXED RECORD

```
student_record = ("Aman", 21, "BCA")
print(student_record)
```

Output:

```
('Aman', 21, 'BCA')
```

Note: Once created, a tuple's values cannot be changed. This makes it ideal in data science for fixed reference data, like column names or configuration values that must never accidentally get modified.

3. Set — unordered, no duplicates allowed

EXAMPLE 1

```
numbers = {1, 2, 3, 3, 2}
numbers.add(4)
print(numbers)
```

Output:

```
{1, 2, 3, 4}
```

EXAMPLE 2 — FINDING UNIQUE CATEGORIES IN DATA

```
cities = ["Delhi", "Mumbai", "Delhi", "Pune", "Mumbai"]
unique_cities = set(cities)
print(unique_cities)
```

Output:

```
{'Delhi', 'Mumbai', 'Pune'}
```

Common set functions: `add()` , `remove()` , `union()` , `intersection()`

4. Dictionary — key-value pairs

EXAMPLE 1

```
student = {"name": "Aman", "age": 21, "course": "BCA"}
print(student["name"])
student["age"] = 22
print(student)
```

Output:

```
Aman {'name': 'Aman', 'age': 22, 'course': 'BCA'}
```

EXAMPLE 2 — DICTIONARY REPRESENTING ONE ROW OF A DATASET

```
record = {"product": "Laptop", "price": 55000, "in_stock": True}
for key, value in record.items():
    print(key, ":", value)
```

Output:

```
product : Laptop price : 55000 in_stock : True
```

Common dict functions: `keys()` , `values()` , `items()` , `get()` , `update()`

Applications

Data structures in Data Science:

- **List:** storing a column of values before converting it into a Pandas Series or NumPy array
- **Tuple:** storing fixed settings like image dimensions (224, 224) in a machine learning model
- **Set:** quickly finding unique values in a column, e.g. unique customer cities
- **Dictionary:** representing a single record/row of data, or mapping category labels to numbers (label encoding)

Quick comparison

Structure	Ordered	Changeable	Duplicates	Symbol
List	Yes	Yes	Yes	[]
Tuple	Yes	No	Yes	()
Set	No	Yes	No	{ }
Dict	Yes (3.7+)	Yes	Keys: No	{key: value}

10. Inbuilt Functions

Python has many built-in functions that are ready to use without importing anything extra. These are used constantly, even inside big data science libraries, for quick calculations and checks.

Function	Use	Example
<code>print()</code>	Displays output	<code>print("Hi")</code>
<code>input()</code>	Takes input from user	<code>input("Enter name: ")</code>
<code>len()</code>	Finds length	<code>len("Hello") → 5</code>
<code>type()</code>	Shows datatype	<code>type(5) → int</code>
<code>int(), float(), str()</code>	Type conversion	<code>int("5") → 5</code>
<code>range()</code>	Generates a sequence of numbers	<code>range(5)</code>
<code>sum()</code>	Adds all items in a list	<code>sum([1,2,3]) → 6</code>
<code>max() / min()</code>	Largest / smallest value	<code>max([3,7,2]) → 7</code>
<code>sorted()</code>	Returns sorted list	<code>sorted([3,1,2]) → [1,2,3]</code>
<code>abs()</code>	Absolute (positive) value	<code>abs(-7) → 7</code>
<code>round()</code>	Rounds a decimal number	<code>round(3.567, 2) → 3.57</code>

EXAMPLE 1 — BASIC DATASET-STYLE SUMMARY

```
nums = [4, 1, 9, 3]
```

```
print(len(nums))
print(sum(nums))
print(max(nums))
print(sorted(nums))
```

Output:

```
4 17 9 [1, 3, 4, 9]
```

EXAMPLE 2 — ROUNDING A CALCULATED STATISTIC

```
values = [10, 20, 30, 25]
average = sum(values) / len(values)
print(round(average, 2))
```

Output:

```
21.25
```

Applications

Inbuilt functions in Data Science:

- `sum()` , `max()` , `min()` for quick statistics before moving to Pandas/NumPy for bigger datasets
- `round()` for presenting clean, readable results (e.g. accuracy scores, percentages)
- `sorted()` for ranking values, like sorting products by sales

Tip: These functions work well for small lists. For real datasets with thousands of rows, the same jobs are usually done faster using Pandas and NumPy.

11. Common Errors

Errors are a normal part of coding, not a sign of failure. Knowing the common ones helps you fix problems quickly instead of getting stuck, which matters a lot when debugging a long data-cleaning script.

Error	Meaning	Example that causes it
SyntaxError	Wrong Python grammar/structure	<code>print("Hi"</code>
NameError	Using a variable that doesn't exist	<code>print(marks)</code>
TypeError	Wrong datatype used in an operation	<code>"5" + 5</code>
ValueError	Right type, but invalid value	<code>int("hello")</code>
IndexError	Accessing an index that doesn't exist	<code>list[10]</code> on a 3-item list
KeyError	Accessing a dictionary key that doesn't exist	<code>student["marks"]</code>
ZeroDivisionError	Dividing a number by zero	<code>10 / 0</code>
IndentationError	Wrong or missing indentation	Missing spaces before a line inside a loop/function

EXAMPLE 1 — TYPEERROR

```
marks = "80"
total = marks + 20
```

Output:

TypeError: can only concatenate str (not "int") to str

EXAMPLE 2 — KEYERROR, COMMON WHILE WORKING WITH DATASET ROWS

```
student = {"name": "Aman", "age": 21}
print(student["marks"])
```

Output:

KeyError: 'marks'

Applications

Why this matters in Data Science:

- **KeyError** often appears when a column name is misspelled or missing in a dataset
- **ValueError** often appears when a "numeric" column secretly contains text, like "N/A"
- **IndexError** often appears when looping through rows and going past the last one
- Reading error messages carefully saves hours of debugging time in real projects

Tip: Always read the last line of the error message first. It usually tells you exactly what went wrong and on which line, so you can fix it quickly instead of guessing.